whether they have voted or not, and the weight of their votes. (Recall that the chairperson is also a voter and that the chairperson's vote counts as double, weight=2.) Let's use this analysis as a guideline and code the two data items identified: `voters` and `proposals`, as shown in listing 3.1. The phases of the voting specified in the problem statement are also coded into an `enum` or enumerated data type. `enum` is an internal data type provided by Solidity. A `Voter` type and `Proposal` type are defined using the `struct` construct, and a special voter, `chairperson`, is also defined. A `mapping` data structure maps the voter account address to the voter details, and an array defines the proposals (numbers) that are being voted on.

Listing 3.1 Data items (BallotV1.sol)

```
pragma solidity >=0.4.2 =<0.6.0;
contract BallotV1 {

    struct Voter {                          ❶
        uint weight;
        bool voted;
        uint vote;
    }
    struct Proposal {                       ❷
        uint voteCount;
    }

    address chairperson;
    mapping(address => Voter) voters;       ❸
    Proposal[] proposals;

    enum Phase {Init, Regs, Vote, Done}     ❹
    Phase public state = Phase.Init;
    }
```

❶ Type Voter contains the voter details.

❷ Type Proposal contains proposal details: for now, only voteCount.

❸ Mapping of voter address to voter details

❹ Various phases (0,1,2,3) of voting, state initialized to Init phase